Lab 5 Advanced: Digital Photo Frame and VGA Game

Submission Due Dates:

Demo:	2025/11/11/ 17:20
Source Code:	2025/11/11/ 18:30
Report:	2025/11/16/ 23:59

Objective

In this lab, you will gain practical experience with **VGA display control**, **block RAM usage**, and **FPGA I/O integration**.
The assignment consists of two major parts:

- **Part A:** Implement a **Digital Photo Frame** with fancy image transition effects.
- **Part B:** Develop a **4×4 Sudoku Game** (數獨遊戲) displayed on the VGA screen.

Both parts will require you to manage image memory, display timing, and user interaction.

Pre-Lab Preparation

- Review the lecture notes on **VGA timing and control**.
- Use **PicTrans.exe** (available in the SampleCode_VGA subfolder) to convert your chosen image into a .coe file.
- Use the provided **template** and **constraint files** for both Part A and Part B.
- Choose your own images for both parts. However, ensure all images are appropriate.

Action Items

Part A: Digital Photo Frame (lab5_1.v, 30%)

Design a VGA controller capable of displaying an image that:

- Scrolls an image across the screen, from the right to the left, the bottom to the top, or the lower-right to the upper-left over time,
- Supports variable scrolling speeds, and
- Allows mirroring image orientation.

a. IO list and Specification:

Signal	I/O	FPGA Pin	Description
clk	Input	connected to W5	100 MHz clock input
rst	Input	connected to U18 (BTNC)	Asynchronous active-high reset
dir_left	Input	connected to V17 (SW0)	Scroll direction control (horizontal)
dir_up	Input	connected to V16 (SW1)	Scroll direction control (vertical)
vmir	Input	connected to W16 (SW2)	Vertical mirror control
hmir	Input	connected to W17 (SW3)	Horizontal mirror control
speed	Input	connected to W15 (SW4)	Scroll speed selection
vgaRed	Output	connected to pin N19, J19, H19, G19	VGA color controls
vgaGreen	Output	connected to pin D17, G17, H17, J17	
vgaBlue	Output	connected to pin J18, K18, L18, N18	
hsync	Output	connected to pin P19	VGA synchronization signals
vsync	Output	connected to pin R19	

- Upon reset (**rst**: the asynchronous positive reset), the VGA display resets and shows the image at the origin position.
- The switches **dir_left** and **dir_up** control the scrolling of the image.
  - If **dir_left** == 1'b0 && **dir_up**== 1'b0: the image holds still on the screen.
  - If **dir_left** == 1'b0 && **dir_up**== 1'b1: the image **scrolls from the bottom to the top** at the frequency of 100MHz divided by 2^{22}.

- If **dir_left** == 1'b1 && **dir_up**== 1'b0: the image **scrolls from the right to the left** at the frequency of 100MHz divided by 2^{22}.

- If **dir_left** == 1'b1 && **dir_up**== 1'b1: the image **scrolls from the lower-right to the upper-left** at the frequency of 100MHz divided by 2^{22}.

- The vertical mirroring switch (**vmir**) and horizontally mirroring switch (**hmir**):
  - If **vmir**==1'b1, the image is mirrored vertically.
  - If **hmir**==1'b1: the image is mirrored horizontally.
  - If both **vmir** and **hmir** are equal to 1'b1, the image is mirrored **both** vertically and horizontally.
  - If both **vmir** and **hmir** are 1'b0, the image remains in its original orientation.

original | mirrored vertically | mirrored horizontally

- The speed switch (**speed**):
  - If **speed**==1'b0, the image will scroll at the frequency of 100MHz divided by 2^{22}.
  - If **speed**==1'b1, the image will scroll at a faster rate of your own choosing. Choose a clearly distinguishable value.
- All signals (**dir_left , dir_up, vmir**, **hmir**, and **speed**) muse operate **independently and concurrently**, regardless of whether other signals are 1'b0 or 1'b1. For example:
  - When the image is paused midway during scrolling, the mirror function should still be applicable and take effect correctly.

		
original	mirrored vertically	mirrored horizontally

  - When the image is mirrored, scrolling should function uninterrupted, maintaining the same scrolling direction.

b. You must use the following template for your design

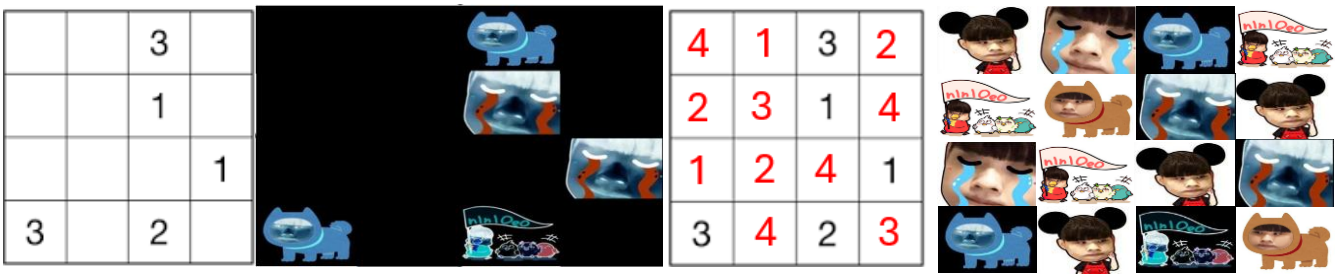
```
module lab5_1 (
  input wire clk,
  input wire rst,
  input wire dir_left,
  input wire dir_up,
  input wire vmir,
  input wire hmir,
  input wire speed,
  output reg [3:0] vgaRed,
  output reg [3:0] vgaGreen,
  output reg [3:0] vgaBlue,
  output reg hsync,
  output reg vsync
  );
  // add your design here
endmodule
```

c. Demo video: https://youtu.be/Z5qbFNONLw4?si=S1EdvIr9t2fSHY7e

Part B: VGA Sudoku Game (lab5_2.v, 70%)

Design a VGA-based controller for a 4x4 Sudoku game (數獨遊戲) with the following features:

- Game States: The game operates through four states: **Init**, **Show**, **Game**, and **Finish**.
- Display Grid: The screen is divided into 4×4 equal-sized grids, with each cell containing an image.
- Choose four **different** pictures that you like. Label these images with the numbers 1 to 4. Then arrange them according to the following rules.
  - **Each row** must contain the numbers 1, 2, 3, and 4, with no repetitions.
  - **Each column** must contain the numbers 1, 2, 3, and 4, with no repetitions.
  - **Each 2×2 box**: The entire 4×4 grid is divided into four 2×2 boxes (i.e., top-left, top-right, bottom-left, bottom-right). Each box must also contain the numbers 1, 2, 3, and 4, with no repetitions.
  - At the beginning of a Sudoku puzzle, some numbers are already filled in. These are called **clues**. They help you get started. Your goal is to complete the entire grid using logic, based on these **clues**. Remember, the **clues** that are already on the grid are fixed — you cannot change them.



clues clue with answers

- When the game state begins, the screen should display the clues in **negative image format** (i.e., each color is inverted to its complementary color). After that, the remaining cells should be filled according to the steps described below.

- First, use the keyboard to select the grid cell you want to fill, based on the encoding shown in the image above. Then, press **Enter** to confirm your selection.

- Second, press a number key from **1 to 4** to select the image you want to insert.
  - **Note:** When a number key (1–4) is entered, the corresponding image should appear in its **normal (positive film) format** in the selected grid cell.
- Next, press **Enter** to confirm the selection. Once confirmed, the cell becomes locked and cannot be changed.
- Repeat the above process until all grid cells are filled. Once completed, the system will automatically enter the **finish state**.
- Design three Sudoku puzzles identical to the examples provides (where black numbers represent the clues and red numbers represent the answers). Use switches SW0-SW2 to select which puzzle to play. Only one switch among SW0-SW2 will be set to 1 at a time, and the desired switch must be configured during the Show state, before entering the Game state.

3	1	4	2
4	2	3	1
1	3	2	4
2	4	1	3

sw0

2	1	4	3
3	4	2	1
1	2	3	4
4	3	1	2

sw1

1	3	2	4
4	2	3	1
2	4	1	3
3	1	4	2

sw2

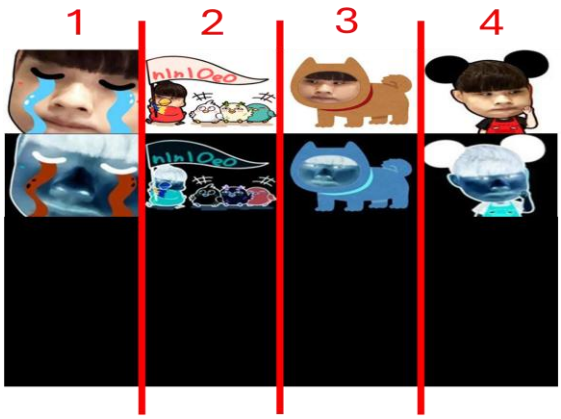
- The two images below illustrate an example pair of positive and negative versions of the same picture.

positive film negative film

a. IO list and Specification:

Signal	I/O	FPGA Pin	Description
clk	Input	connected to W5	100 MHz clock input
rst	Input	connected to U18 (BTNC)	Asynchronous active-high reset
start	Input	Connected to T17 (BTNR)	Control the state transition
sw[2:0]	Input	connected to SW0-SW2	Select which of the three Sudoku games to display.
PS2_CLK	Inout	connected to C17	PS2 Keyboard clock
PS2_DATA	Inout	connected to B17	PS2 Keyboard data
vgaRed	Output	connected to pin N19, J19, H19, G19	VGA color controls
vgaGreen	Output	connected to pin D17, G17, H17, J17	
vgaBlue	Output	connected to pin J18, K18, L18, N18	
hsync	Output	connected to pin P19	VGA synchronization signals
vsync	Output	connected to pin R19	
led[15:0]	Output	connected to LD15~LD0	Indicates result of the game.

- Upon reset (**rst**: the asynchronous active-high reset), the VGA display will show a completely black screen.
- The **start** button is used to control the gameplay (for the state transitions).
- **Init state**:
  - All grid blocks are displayed as **black**.
  - All LEDs are **off**.
  - Pressing the **start** button transition to the **Show** state.
- **Show state**:



- Display the four selected images in the order of their assigned codes.
- The **first row** should show the images in **positive** film (normal colors).
- The **second row** should show images in **negative** film (complementary colors).
- Everything below these two rows should remain **black**. (Refer to the illustration above for the **show state** layout.)
- All LEDs remain **off** during this state.
- Pressing the **start** button transition to the **Game** state.

- **Game state**:
  - At the beginning of this state, the clue cells should be displayed in negative image format.
  - Players interact with the game as follows:
    i. Select a grid cell using the keyboard. Then press **Enter** to confirm the selection.
       - If multiple keys are pressed before Enter, the last key pressed determines the selected cell.
    ii. Press a number key from **1 to 4** to choose the corresponding image. Then press **Enter** to confirm the placement.
       - If multiple numbers are pressed before Enter, the last number pressed determines the selected image.
       - Once confirmed, the cell becomes locked and cannot be changed.
  - Previously placed images cannot be overwritten. And original clue cells remain fixed.
  - All LEDs remain **off** during this state.
  - Once all cells are filled, the game automatically transitions to the **Finish state**.
- **Finish state**:

- For each grid cell, if the answer is correct (according to the reference solution), the corresponding LED lights up.
- LEDs for original clue cells also lights up.
- All the grid cells are locked and cannot be modified.
- Pressing the **start** button transitions to the **Init** state.

- **Hint**: There are two common methods to display a 4x4 grid of image blocks:
  i. Composite Image Approach: Create one large image that includes all 16 smaller images in the desired layout.
  ii. Image Mapping Approach: Store four individual images and implement a display converter to arrange them dynamically on the VGA screen.

b. Grading:

Function	Score (70%)
IDLE state	5%
SHOW state	10%
GAME state	45%
FINISH state	10%

c. You must use the following template for your design

```
module lab5_2 (
  input wire clk,
  input wire rst,
  input wire start,
  input wire [2:0] sw,
  inout wire PS2_CLK,
  inout wire PS2_DATA,
  output reg [3:0] vgaRed,
  output reg [3:0] vgaGreen,
  output reg [3:0] vgaBlue,
  output reg hsync,
```

```
    output reg vsync,
    output reg [15:0] led
);
    // add your design here
endmodule
```

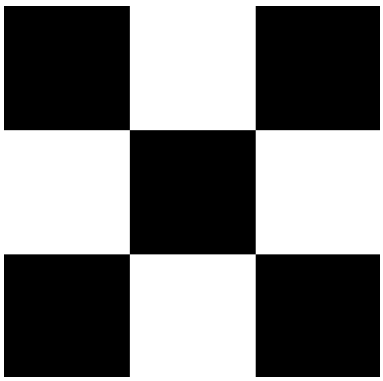
d. Demo video: https://www.youtube.com/watch?v=cS5veIPaOVY

Questions and Discussion

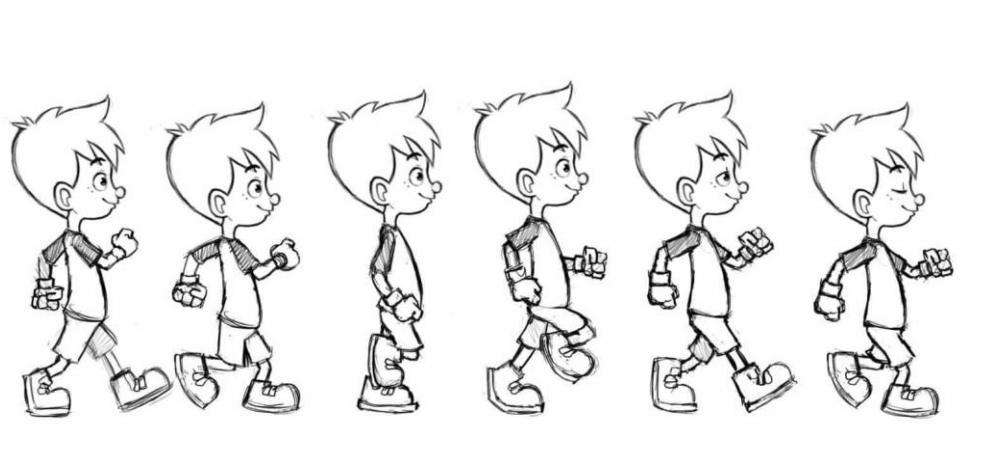
Please answer the following questions in your report.

A. Our FPGA equips with the BRAM of only 1800 Kbits, which a 640×480 image cannot fit in. If we want to implement a video game, apart from storing a smaller image (e.g., 320×240) as we did in this lab, please give at least 2 possible methods to reduce the BRAM usage.

B. Verilog supports 2D arrays. Briefly explain how you may use it to generate the following picture without using the RAM.



C. Suppose that you are given Picture A below and are asked to make an animation of the little boy walking in the middle of the screen. Describe your approach to implement the animation? Provide pseudo-code or concept logic with an explanation.



(Picture A)

Report Guidelines

Your report should include but not limit to the following items.

A. Lab Implementation (35%)
   1. Block diagram of the design with explanation.
   2. Description of each essential block.
   3. State diagram(s) for FSM(s) with explanation, if applicable.

B. Questions & Discussions (50%)
   Provide your answer to the Questions & Discussions in the lab assignment.

C. Problem Encountered (10%)
   Describe the problems you encountered, solutions you developed, and the discussion. Explaining them with code segments or diagrams is recommended.

D. Suggestions (5%)
   Optional: Feedback on the lab, course suggestions, or even a light-hearted joke.

Attention

- Please use the same template modules of keyboard_decoder, KeyboardCtrl, Ps2Interface in Lab 4.
- Hand in **two Verilog files**: lab5_1.v, lab5_2.v.
- Merge all modules into a single .v file for each part.
- Do **not** include modules such as vga_controller, keyboard_decoder, KeyboardCtrl, onepulse, and debounce in your submission.
- Do **not** submit compressed ZIP files, which will be considered an incorrect format.
- Report filename format: **lab5_report_StudentID.pdf** (e.g., lab5_report_123456789.pdf).

- You have the responsibility to check if the submission is successful.
- DO NOT submit ZIP files, which will be considered as incorrect format.
- Prepare bitstream files before demo to avoid delays.
- Be ready to answer TA questions during demo.
- Post questions on EECLASS forum if unclear.